

IIT Madras Zanzibar

RecipeVault

A Dessert Recipe Assistant Powered by a RAG Pipeline

Milestone 3

Performance Evidence Report

Hemamalini Kamaladhasan

Roll Number: ZDA24B025

Z2004: Database Management Systems

Track A: RAG Pipeline

1. Project Summary

RecipeVault is a dessert and baking recipe assistant built on a relational PostgreSQL database. It is designed for Track A - a Retrieval-Augmented Generation (RAG) pipeline that answers natural-language questions such as “give me a chocolate cake recipe under five steps” by retrieving grounded results directly from the database and returning them with their source.

The dataset began as the RecipeNLP collection of 100,000 recipes from Kaggle. I narrowed this down specifically to dessert and baking recipes using title-based keyword filtering, producing a focused and coherent dataset of 1,755 recipes. I also enriched the data with two custom columns that did not exist in the original source: `occasion` (Everyday, Christmas, Birthday, Halloween, Valentine, Easter) and `flavour_profile` (Chocolate, Vanilla/Cream, Citrus, Caramel, Fruity/Berry, Spiced, Nutty, Tropical). These columns are central to how the system filters and retrieves recipes.

This report focuses on the performance layer of the project. It identifies the queries that matter most to the RAG pipeline, explains why they were slow, applies B-Tree indexes to optimise them, and presents before and after execution plan evidence captured using `EXPLAIN ANALYZE`.

2. Objective

The objective of this milestone is to demonstrate measurable query optimisation. Specifically, this report aims to:

- Identify the queries most frequently used by the RecipeVault retrieval layer.
- Explain why these queries are expensive when run against an unindexed database.
- Design and justify appropriate B-Tree indexes for the relevant columns.
- Provide before and after `EXPLAIN ANALYZE` evidence proving the improvement.
- Document a stored procedure that supports the retrieval layer.

3. Test Environment

All measurements were taken on the same machine under the same conditions to keep the comparison fair. The environment is summarised below.

Item	Detail
Database engine	PostgreSQL 18.3 (Homebrew)
Machine	Apple MacBook Pro, Apple Silicon (aarch64)
Operating system	macOS (Darwin 25.2)
Recipes	1,755 rows
Recipe–ingredient links	17,015 rows
Cooking steps	27,532 rows
NER tags	34,070 rows

4. Queries Identified for Optimisation

Two queries were chosen because they sit directly on the critical path of the RAG pipeline where they run every time a user asks for a recipe.

Query 1 - Filter recipes by flavour profile

This is the single most frequent query in the system. Whenever a user asks for a recipe of a particular flavour for example a chocolate dessert, the retrieval layer filters the recipe table by `flavour_profile`. Without an index, PostgreSQL must read all 1,755 rows sequentially and discard the 1,300 that do not match, even though only 455 are needed.

Query 2 - Join recipes with NER tags by occasion

This query powers seasonal retrieval. For example, fetching every Christmas recipe together with its extracted ingredient tags. Without indexes, PostgreSQL performs a Hash Join that scans all 34,070 NER tag rows and all 1,755 recipe rows, despite the final result containing only 262 rows. This is the most expensive query in the system and therefore the most valuable to optimise.

5. Methodology

Each query was executed first against the unindexed database to establish a baseline, then re-executed after the relevant B-Tree indexes were created. Each query was run several times and the representative execution time recorded, so that one-off caching effects did not distort the comparison. The same dataset, same machine, and same session settings were used throughout. All timing and plan data was captured with the command `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)`, which reports both the chosen execution plan and the real measured execution time.

6. Results

Query 1 - flavour_profile filter

Metric	Before Index	After Index
Scan type	Sequential Scan	Bitmap Index Scan
Execution time	1.140 ms	0.354 ms
Rows read	1,755 (all rows)	455 (matching only)
Improvement	Baseline	3.2× faster

Before indexing, PostgreSQL read every row and filtered 1,300 of them out. After indexing, it used `idx_recipe_flavour` to locate the 455 matching rows directly, avoiding the wasted reads.

Query 2 - occasion + NER tag join

Metric	Before Index	After Index
Scan type	Hash Join + Seq Scan	Nested Loop + Index Scans
Execution time	9.421 ms	0.191 ms
Rows read	35,825 (both tables)	275 (indexed lookups)
Improvement	Baseline	49× faster

This is the headline result of the report. Before indexing, PostgreSQL scanned all 34,070 NER tag rows. After indexing, it used `idx_recipe_occasion` to find the 13 Christmas recipes first, then used `idx_ner_tag_recipe_id` to fetch only the relevant tags for those recipes reducing execution time from 9.421 ms to 0.191 ms, a 49-fold improvement.

7. Index Design and Justification

Five B-Tree indexes were created. B-Tree is the PostgreSQL default and the correct choice here because every target column is used either in an equality filter (`WHERE flavour_profile = 'Chocolate'`) or as a join key (`ON recipe_id = recipe_id`). B-Tree indexes handle equality and range lookups efficiently and perform well on the low-to-medium cardinality columns used in this project.

```
CREATE INDEX idx_recipe_flavour      ON recipe(flavour_profile);
CREATE INDEX idx_recipe_occasion    ON recipe(occasion);
CREATE INDEX idx_ner_tag_recipe_id  ON ner_tag(recipe_id);
CREATE INDEX idx_step_recipe_id     ON step(recipe_id);
CREATE INDEX idx_recipe_ingredient_recipe_id ON recipe_ingredient(recipe_id);
```

The `flavour_profile` and `occasion` indexes accelerate the two filter queries above. The three foreign-key indexes (`recipe_id` on `ner_tag`, `step`, and `recipe_ingredient`) accelerate the joins used throughout the analytical and retrieval queries, since every one of those joins links a child table back to the recipe table.

8. Stored Procedure

A stored procedure, `get_recipes_by_flavour()`, was written to support the retrieval layer. It takes a flavour profile as input and returns every matching recipe along with its occasion and total step count, ordered from most to least complex. The RAG pipeline calls this procedure to assemble candidate recipes before passing them to the language model.

```
SELECT * FROM get_recipes_by_flavour('Chocolate') LIMIT 5;
```

Sample output:

ID	Title	Occasion	Steps
1154	Cannoli	Everyday	84
52	Chocolate Chip Cookies	Everyday	84
1993	White Chocolate Stick Cake	Everyday	84

ID	Title	Occasion	Steps
980	Chocolate Cupcakes with Mascarpone Filling	Everyday	76
1496	Chocolate Caramel Tart with Sea Salt	Everyday	74

9. Interpretation of the Query Plans

The decisive difference between the before and after plans is the scan type chosen by the query planner. A Sequential Scan reads every row in a table regardless of how many actually match the condition. An Index Scan instead follows pointers in the index structure and reads only the rows that satisfy the condition. A Bitmap Index Scan sits between the two. It reads the index to build a bitmap of the pages that contain matches, then fetches only those pages from the table.

For Query 2 the more important change is the join strategy. A Hash Join builds a hash table from one input and probes it with the other; this is efficient when many rows match, but wasteful when only a handful do. Once the occasion index existed, the planner switched to a Nested Loop. It took the 13 Christmas recipes from the outer table and, for each one, used the NER tag index to fetch just that recipe's tags. Because the outer set was tiny, the indexed Nested Loop was far cheaper than scanning 34,070 rows, which is exactly why the execution time collapsed from 9.4 ms to 0.2 ms.

10. Conclusions

Both queries improved measurably after indexing, and Query 2 in particular improved by a factor of 49. The results confirm that placing B-Tree indexes on foreign-key columns and on frequently filtered columns substantially reduces query cost even on a relatively modest dataset of 1,755 recipes. On a larger production dataset the gap between an indexed and an unindexed query would widen further, since sequential-scan cost grows linearly with table size while indexed-lookup cost grows only logarithmically. For RecipeVault, these indexes mean the retrieval layer can return candidate recipes to the RAG pipeline almost instantly, keeping the overall question-answering experience responsive.

AI Usage Disclosure

Claude (Anthropic) was used for syntax suggestions, debugging help, and structuring this report. All queries were written, executed, verified, and interpreted independently, and every result presented here was produced on my own machine.